

Laboratori de GIA-ABIA

Sessió 6: Algoritmes genètics

En aquesta sessió veurem com desenvolupar i executar algoritmes genètics.

Preparació de l'entorn

Definirem una representació de la solució basada en bits i per tant utilitzarem el mòdul de Python `bitarray`.

Primer activeu el vostre entorn virtual (com s'explica a la sessió 1). Un cop activat:

```
1 pip install bitarray
```

Elements del problema

Reciclem la definició del problema que vam utilitzar per hill climbing i simulated annealing:

```
1 class ProblemParameters(object):
2     def __init__(self, h_max: int, v_h: List[int], p_max: int,
3         c_max: int):
4         self.h_max = h_max # Alcada maxima de tots els
5         self.v_h = v_h     # Al ades de cada paquet
6         self.p_max = p_max # Maxim numero de paquets
7         self.c_max = c_max # Maxim numero de contenidors
8
9     def __repr__(self):
10        return f"Params(h_max={self.h_max}, v_h={self.v_h},
11            p_max={self.p_max}, c_max={self.c_max})"
```

Per tant, podeu utilitzar el mateix fitxer que ja utilitzavem en sessions anteriors (`bin_packing_problem_parameters.py`).

Algorisme

A `search.py` teniu una implementació senzilla d'un algorisme genèric que permet definir representacions arbitràries (per exemple, permet definir solucions amb enters o strings). Aquesta implementació no obstant té diversos inconvenients:

- No és molt eficient (ja que ha de permetre representacions arbitràries, no s'optimitza l'estructura de dades).
- No permet redefinir els operadors de encreuament i mutació i només inclou operadors per defecte totalment aleatoris.
- No es pot configurar la probabilitat d'encreuament.

Per tant us proposem aquí una implementació ad hoc que s'ajusta a l'explicació feta a teoria. **La podeu trobar al fitxer `bin_packing_genetic_algorithm.py` del web de l'assignatura.**

La nostra representació de la sol·lució serà un vector de bits que representarà una matriu de `c_max` vectors de mida `p_max`. A cada vector `j`, el seu bit `i` indicarà si el paquet `p_i` està assignat al contenidor `c_j`.

Per exemple, si tenim `c_max = 3` i `p_max = 2` i els dos paquets estan assignats al primer contenidor, tindrem la representació:

```
1 11 00 00
```

Si tenim `c_max = 5` i `p_max = 5` i el primer paquet va en el primer contenidor, el segon paquet en el segon contenidor, etc., tindrem la representació:

```
1 10000 01000 00100 00010 00001
```

Funcions d'avaluació, encreuament i mutació

Anem a definir ara exemples de funcions d'avaluació, encreuament i mutació. **Els podeu trobar implementats al fitxer `bin_packing_genetic_functions.py` del web de l'assignatura.**

La funció d'avaluació que proposem té dos components:

- l'entropia (negada, ja que volem maximitzar) elevada al quadrat (per donar més importància a valors més propers a 0) i
- una penalització per condicions que invaliden la sol·lució:
 - si un contenidor té overflow, penalitzem 20 per cada unitat d'alçada d'overflow
 - si un paquet no està assignat a un (i només un) contenidor, penalitzem 50 per cada contenidor de més o de menys

Podeu jugar amb aquestes constants de penalització a la seva declaració a dins de la funció:

```

1 def fitness_function(individual: bitarray, params:
    ProblemParameters) -> float:
2     # Constants per ponderar les penalitzacions
3     OVERFLOW_PENALTY = 20
4     WRONG_ASSIGNMENT_PENALTY = 50
5
6     [...]

```

Per l'operador d'encreuament, en comptes de creuar per un punt aleatori, ho farem assegurant que no tallem vectors de contenidor pel mig:

```

1 def cross_function(a: bitarray, b: bitarray, params:
    ProblemParameters) -> List[bitarray]:
2     cross_container = random.randint(0, params.c_max - 1)
3     cross_point = cross_container * params.p_max
4     c = bitarray()
5     d = bitarray()
6     for i in range(cross_point):
7         c.append(a[i])
8         d.append(b[i])
9     for i in range(cross_point, len(a)):
10        c.append(b[i])
11        d.append(a[i])
12    return [c, d]

```

A l'operador de mutació farem que només esculli un bit aleatori i el giri:

```

1 def mutate_function(a: bitarray, _: ProblemParameters) ->
    bitarray:
2     array_length = len(a)
3     mutate_point = random.randint(0, array_length - 1)
4     a.invert(mutate_point)
5     return a

```

Provant l'algoritme

Definirem un problema amb paràmetres petits: 5 contenidors, 5 paquets, alçada màxima 5, paquets amb alçades de l'1 al 5. Per començar també escollirem valors baixos pels paràmetres de l'algorisme:

- 10 generacions
- 10 individus per generació

- Probabilitat d'encreuament 20%
- Probabilitat de mutació 5%

```

1 params = ProblemParameters(5, [i + 1 for i in range(5)], 5, 5)
2 algorithm = BinPackingGeneticAlgorithm(params, 100, 0.2, 0.05,
    fitness_function, cross_function, mutate_function, 42)
3 population = algorithm.generate_initial_population()
4 solutions = algorithm.run_algorithm(100)

```

Exercicis proposats

- Llegiu detingudament i analitzeu el mètode `run_algorithm()` implementat a `BinPackingGeneticAlgorithm`. Identifiqueu les diferents fases de cada iteració. S'ajusta a l'algoritme que heu vist a classe de teoria?
- Proveu a fer un main que executi l'algoritme amb els paràmetres proposats en la secció anterior. Us sembla que el resultat és bo?
- Intenteu modificar:
 - els paràmetres de les probabilitats dels operadors,
 - el número d'iteracions,
 - el tamany de la població, i/o
 - les constants de penalització de la funció d'avaluació

per millorar el resultat. Quines combinacions us funcionen millor? Podeu aconseguir una solució òptima (tres contenidors)?

- Intenteu trobar funcions d'avaluació, encreuament i mutació per millorar el vostre resultat encara més.
- L'estratègia de generació de sol·lució inicial es troba al mètode de la classe `BinPackingGeneticAlgorithm`: `generate_initial_solution()`. Tal com podeu veure, la implementació que us donem es basa en poblar amb bits totalment aleatoris, sense cap criteri. Se us acut una altra estratègia que ajudi a convergir més ràpidament en una bona sol·lució?
- Un cop tingueu unes probabilitats i unes funcions que us convencin, proveu a escalar el problema, e.g. 50 paquets i 50 contenidors.